



به نام خدا

سیستم عامل - بهار ۱۴۰۴

استاد: دکتر مهدی کارگهی

پروژه ۳: طراحی چند ریشه‌ای



طراحان: علی قنبری - مهدی حاجی سید حسین

مقدمه

در این پروژه، قصد داریم یک شبکه عصبی مصنوعی را با استفاده از برنامه‌نویسی موازی و تکنیک‌های همگام‌سازی پیاده‌سازی کنیم. هدف اصلی، آشنایی با مفاهیم برنامه‌نویسی چند نخه^۱ و مکانیزم‌های همگام‌سازی در سیستم عامل می‌باشد.

شرح پروژه

مروری بر شبکه‌های عصبی

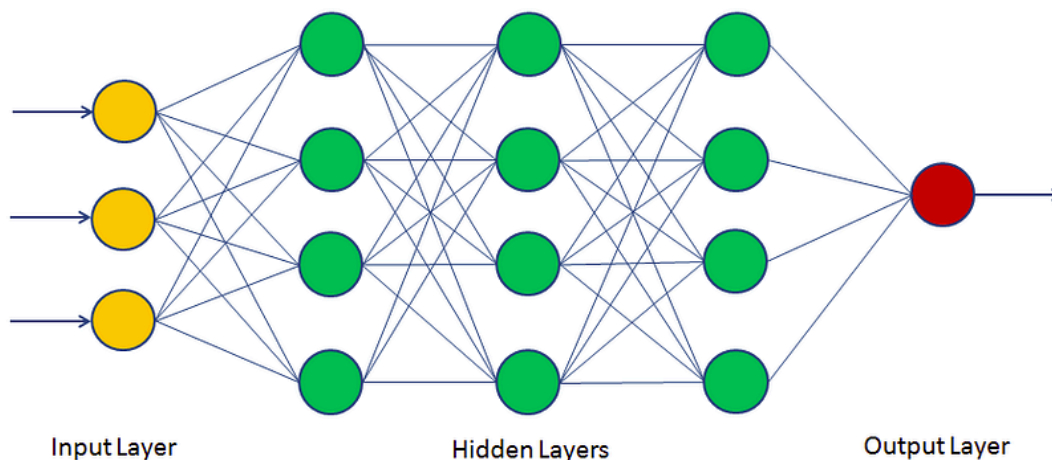
شبکه‌های عصبی مصنوعی مدل‌های محاسباتی هستند که از ساختار و عملکرد سیستم عصبی انسان الهام گرفته‌اند. یکی از پرکاربردترین این مدل‌ها، شبکه عصبی پرسپترون چندلایه^۲ است که از یک لایه ورودی، یک یا چند لایه میانی (پنهان) و یک لایه خروجی تشکیل شده است.

ساختار شبکه عصبی پرسپترون چندلایه

در این نوع شبکه، جریان داده‌ها فقط در یک جهت و از لایه ورودی به سمت لایه‌های پنهان و از آنجا به لایه خروجی حرکت می‌کند (معماری پیش‌ران یا Feed-Forward).

^۱ Multi-Thread

^۲ Multilayer perceptron



شکل ۱: شمای یک شبکه عصبی پرسپترون چندلایه و جریان داده در آن

هر نورون در لایه‌های میانی و خروجی، خروجی تمام نورون‌های لایه قبل را دریافت کرده و مجموع حاصل ضرب آنها در وزن‌های مخصوص به خود را محاسبه می‌کند. سپس یک تابع فعال‌سازی^۳ بر روی این مجموع اعمال می‌شود تا خروجی نهایی نورون تولید شود. این خروجی به تمام نورون‌های لایه بعد ارسال می‌گردد.

$$y = \sigma(\sum_j w_j x_j + b)$$

- x_j ورودی‌های نورون
- w_j وزن‌های متناظر با هر ورودی
- b مقدار بایاس نورون
- σ تابع فعال‌سازی
- y خروجی نورون

توابع فعال‌سازی متداول را می‌توانید از [اینجا](#) ببینید.

مجموعه داده MNIST

مجموعه داده MNIST یکی از پرکاربردترین مجموعه داده‌های آموزشی در زمینه یادگیری ماشین و به خصوص شبکه‌های عصبی است. این مجموعه داده شامل تصاویر ارقام دست‌نویس 0 تا 9 با ابعاد

^۳ Activation Function

28×28 پیکسل است که توسط Yann LeCun و همکارانش جمع‌آوری و منتشر شده است. برای شناسایی ارقام دست‌نویس MNIST، یک شبکه عصبی با ساختار زیر طراحی شده است:

- **لایه ورودی:** 784 نورون (28×28 پیکسل تصویر)
- **لایه میانی:** 256 نورون با تابع فعال‌سازی ReLU
- **لایه خروجی:** 10 نورون (برای اعداد 0 تا 9) با تابع فعال‌سازی Sigmoid



شکل ۲. تصاویر نمونه از مجموعه داده تست MNIST

پیاده‌سازی سریال شبکه عصبی

قبل از پرداختن به پیاده‌سازی موازی، لازم است با نسخه سریال و ترتیبی الگوریتم آشنا شویم. در پیاده‌سازی سریال، مراحل زیر به صورت متوالی انجام می‌شوند:

1. **خواندن داده‌ها:** خواندن تصاویر و برچسب‌های آنها از فایل‌های مجموعه داده MNIST
2. **پیش‌پردازش:** تبدیل داده‌ها به فرمت مناسب برای شبکه عصبی
3. **محاسبات لایه میانی:** برای هر نورون لایه میانی، ضرب ورودی‌ها در وزن‌ها و اعمال تابع فعال‌سازی
4. **محاسبات لایه خروجی:** انجام همین محاسبات برای لایه خروجی
5. **پیش‌بینی:** تعیین بیشترین مقدار خروجی به عنوان عدد پیش‌بینی شده
6. **ارزیابی:** مقایسه پیش‌بینی با برچسب واقعی و محاسبه دقت

به همراه صورت پروژه فایلی با نام `serial.cpp` به شما داده شده است که در آن این شبکه عصبی بصورت سری پیاده سازی شده است.

پیاده سازی موازی شبکه عصبی

در این پروژه، هدف پیاده سازی موازی شبکه عصبی با استفاده از نخ‌ها⁴ می باشد. ابعاد مختلف موازی سازی در این پروژه شامل:

1. موازی سازی در سطح لایه: نخ‌های مختلف برای لایه‌های مختلف شبکه
2. موازی سازی در سطح نورون: نخ‌های متعدد برای محاسبه خروجی نورون‌های مختلف در یک لایه

ساختار شبکه عصبی موازی

شبکه‌ی عصبی از 1 نخ برای لایه‌ی ورودی، 8 نخ برای لایه‌ی میانی اول، 10 نخ برای لایه‌ی خروجی و 1 نخ برای ارائه‌ی عدد خروجی شبکه، که عدد پیش‌بینی شده توسط شبکه را به دست می‌دهد، تشکیل می‌شود. وزن‌ها و بایاس مربوط به هر نورون در لایه‌های میانی در پوشه `net_params` قرار دارد.

لایه ورودی

در این لایه، یک نخ مسئول خواندن پیکسل‌های یک تصویر از فایل `t10k-images-idx3-ubyte`، که در داخل پوشه `data` قرار دارد، می‌شود. از آنجایی که تعداد کل تصاویر 10000 است، این نخ 10000 بار این کار را تکرار می‌کند.

لایه میانی

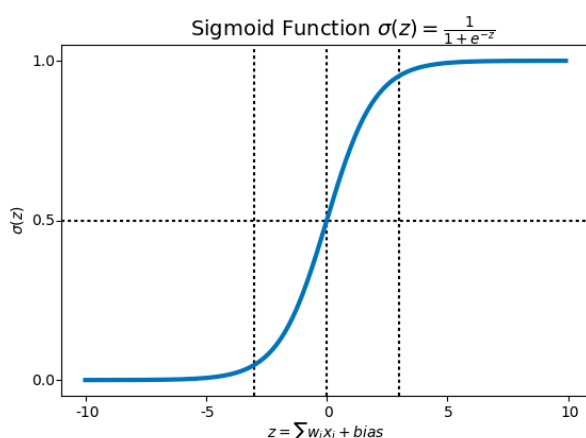
در این لایه 256 نورون وجود دارد که هر نورون دارای پارامترهای مخصوص به خود (وزن‌های متناظر با هر پیکسل تصویر و بایاس نورون) است. از 8 نخ برای انجام محاسبات مربوط به هر نورون استفاده می‌کنیم که به این ترتیب هر نخ، وظیفه‌ی انجام محاسبات مربوط به 32 نورون را بر عهده دارد. تابع فعال‌ساز هر نورون را در این لایه Relu در نظر بگیرید.

⁴ Threads

$$RELU(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

لایه خروجی

در این لایه نیز 10 نورون، متناظر با اعداد 0 تا 9، وجود دارد که از 10 نخ، به ازای هر نورون یک نخ، برای پیاده‌سازی آن استفاده می‌کنیم. فعال‌سازِ نورون‌های این لایه را تابع سیگموید⁵ در نظر بگیرید:



عدد خروجی شبکه

در انتها، یک نخ لایه‌ی خروجی عدد بیش‌بینی شده توسط شبکه را به‌دست می‌آورد. برای این کار، مقدار ماکزیمم خروجی‌های لایه‌ی قبل را محاسبه کرده و عدد متناظر با آن مقدار را به عنوان عدد بیش‌بینی شبکه در نظر بگیرید. سپس این عدد با عدد اصلی متناظر با آن تصویر، که در فایل `t10k-labels-idx1-ubyte` قرار دارد، مقایسه می‌کند و نرخ موفقیت شبکه را به‌روز می‌کند.

مکانیزم‌های همگام‌سازی

۱. اولویت‌بندی نخ‌ها (Thread Prioritization)

اضافه کردن سیستم اولویت برای نخ‌ها، که نخ‌های لایه‌های مختلف، دسترسی متفاوتی به منابع داشته باشند:

- **اولویت بالا:** نخ‌های لایه خروجی (برای سرعت پاسخ‌دهی)

⁵ Sigmoid function

- اولویت متوسط: نخ‌های لایه‌های میانی
- اولویت استاندارد: نخ‌های لایه ورودی و پیش‌پردازش

([لینک کمکی](#) برای این بخش)

۲. سمافورها و موانع (Semaphores and Barriers)

برای انجام صحیح محاسبات لازم است سمافورهایی در بین لایه‌های این شبکه تعبیه شوند که انتقال داده‌ها را در میان این لایه‌ها مدیریت کنند. نحوه‌ی پیاده‌سازی سمافورها از تنوع بسیاری برخوردار است و استفاده از کمترین تعداد سمافورها مبنای کار است.

جزئیات پیاده‌سازی

پارامترهای شبکه عصبی

- وزن‌ها و بایاس‌های مربوط به هر نورون در پوشه `net_params` قرار دارند.
- ساختار فایل‌های وزن باید با معماری شبکه همخوانی داشته باشد.

پردازش داده

- فایل‌های داده در پوشه `data` شامل:
 - `t10k-images-idx3-ubyte`: تصاویر آزمایشی
 - `t10k-labels-idx1-ubyte`: برچسب‌های آزمایشی

بخش امتیازی

- اجازه دهید تعداد لایه‌های میانی و تعداد نورون‌های هر لایه در زمان اجرا و به صورت دلخواه تعیین شود.
- استخر نخ^۶ با قابلیت تخصیص پویای نخ‌ها بر اساس بار محاسباتی پیاده‌سازی شود. (می‌توانید از [این لینک](#) برای پیاده‌سازی کمک بگیرید)

^۶ Thread Pool

نکات و نحوه تحویل

- کد شما باید به زبان C++ نوشته شود و با کامپایلر g++ در سیستم عامل لینوکس قابل کامپایل باشد.
- دقت شود برای موازی‌سازی پروژه تنها مجاز به استفاده از کتابخانه **Pthread** هستید و استفاده از کتابخانه‌های دیگر به جز کتابخانه‌های پایه زبان C++، مجاز نیست. (برای مطالعه توابع مهم این کتابخانه می‌توانید از [این لینک](#) استفاده کنید)
- توجه کنید کد شما نباید دارای وضعیت بن بست^۷، شرایط مسابقه^۸ و سایر مشکلات پیاده‌سازی چندنخی باشد.
- فایل آپلودی شما باید به فرمت OS-CA3-<SID>.zip باشد و speedup حاصل شده (در نسخه موازی نسبت به نسخه سریال) باید در آن گزارش شده باشد.
- در پیاده‌سازی خود در خصوص مواردی که ذکر نشده‌اند، می‌توانید فرض منطقی و دلخواه خود را داشته باشید.
- در صورت داشتن سوال می‌توانید از طریق فروم درس یا شرکت در جلسات رفع اشکال سوالات خود را مطرح کنید.

موفق باشید.

^۷ Deadlock

^۸ Race Condition